

Python for Quant Interviews: The Complete Guide

From Scripting to System Architecture
Speed, Memory, and System Design Made Practical

Amit Kumar Jha, Quant Analyst at UBS

January 18, 2026

Contents

Module 0: Python Fundamentals Refresher	3
0.1 The 10 Python Numbers Every Quant Must Know	4
0.2 Variables and Operators	4
0.3 Control Flow Patterns	4
0.4 OOP Fundamentals	5
0.5 File I/O for Quants	6
0.6 Exception Handling	7
0.7 String Operations and Regex	8
Module 1: Types as Financial Objects	9
1.1 The Financial Type System	9
1.2 float = Price (The Danger Zone)	10
1.3 list = Time Series	10
1.4 dict = Portfolio	11
1.5 function = Derivative Contract	12
1.6 class = Financial Instrument	12
Module 2: Memory and the GIL	14
2.1 The Global Interpreter Lock (GIL)	14
2.2 When to Use Threading vs Multiprocessing	15
2.3 Memory Model: Everything is an Object	15
2.4 __slots__: The Memory Saver	16
Module 3: Vectorization — The Speed Secret	17
3.1 Why Loops Are Slow	17
3.2 Benchmark: Loop vs Vectorized	18
3.3 Views vs. Copies: The Silent Bug	18
3.4 Broadcasting: Implicit Vectorization	19
Module 4: High-Performance Python (JIT Compilation)	22
4.1 What is JIT Compilation?	22
4.2 Monte Carlo Example: 100x Speedup	22
Module 5: Quant System Architecture	24
5.1 Event-Driven vs. Vectorized Backtesting	24
5.2 The Event Loop Pattern	25
5.3 AsyncIO for Market Data	26
Module 6: Python Gotchas (Trick Questions)	27

```

8 for t in threads: t.start()
9 for t in threads: t.join()

```

Expected: 8x speedup.

Actual: No speedup (slightly slower due to context switching overhead).

Why? The GIL forces all 8 threads to take turns executing Python code. Only one runs at any moment.

2.2 When to Use Threading vs Multiprocessing

Task Type	Use	Why
CPU-Bound (Pricing, Math)	multiprocessing	Separate memory spaces, no GIL
I/O-Bound (Network, Disk)	threading or asyncio	GIL released while waiting

Practitioner Insight

The Production Pattern: Process Pool

For CPU-heavy tasks like pricing 10,000 options:

```

1 from multiprocessing import Pool
2
3 def price_one_option(params):
4     # Heavy calculation
5     return black_scholes(*params)
6
7 if __name__ == "__main__":
8     with Pool(processes=8) as pool:
9         results = pool.map(price_one_option, list_of_params)

```

Why this works:

- Each process has its own Python interpreter and GIL.
- True parallelism on 8 cores.
- Data is serialized (pickled) between processes—overhead exists but worth it.

Interview: “For CPU-bound tasks, I use multiprocessing.Pool. The overhead of pickling data is small compared to the speedup from true parallelism. For I/O-bound tasks like fetching market data, I use asyncio.”

“The if __name__ == “__main__” Guard”

On Windows, multiprocessing spawns new processes that re-import the module. Without the guard, you get infinite recursion. **Always** wrap Pool code in this block.

2.3 Memory Model: Everything is an Object

Python’s Memory Overhead

In Python, **everything is an object**. Even an integer.

```

1 import sys
2 sys.getsizeof(42) # 28 bytes (!)

```

A C int is 4 bytes. A Python int is 28 bytes because it stores:

- Type pointer (8 bytes)
- Reference count (8 bytes)
- Value (8 bytes)
- Other metadata

Module 4: High-Performance Python (JIT Compilation)

Module Overview

What this module is about:

When NumPy isn't enough (path-dependent simulations, complex conditionals), use Just-In-Time (JIT) compilation to reach C++ speeds.

Why this matters:

Some algorithms cannot be vectorized (e.g., path-dependent exotic options). JIT lets you write Python but execute machine code.

After this module: You will know when and how to use Numba.

4.1 What is JIT Compilation?

Compile at Runtime

Normally, Python is *interpreted*: code is translated line-by-line at runtime.

A JIT compiler translates your function into **machine code** the first time it runs. Subsequent calls execute the compiled code directly.

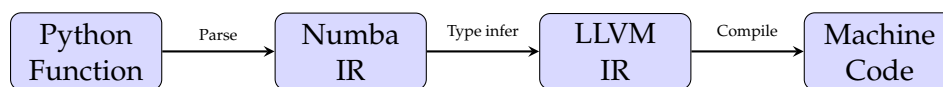
Numba does this for a subset of Python/NumPy:

```
1 from numba import jit
2
3 @jit(nopython=True)
4 def fast_function(x):
5     # This is compiled to machine code
6     ...
```

Core Formula

`@jit(nopython=True)` = "Compile or fail"

Numba Compilation Pipeline



First call: compile. Subsequent calls: execute directly.

`nopython=True` means: "Do not fall back to the Python interpreter. If you can't compile this, raise an error."

4.2 Monte Carlo Example: 100x Speedup

```
1 import numpy as np
2 from numba import jit
3 import math
4
5 # Pure Python (SLOW)
6 def bs_mc_python(S0, K, T, r, sigma, n_sims, n_steps):
7     dt = T / n_steps
8     total = 0.0
9     for _ in range(n_sims):
10         st = S0
11         for _ in range(n_steps):
12             z = np.random.normal()
13             st = st * math.exp((r - 0.5*sigma**2)*dt + sigma*math.sqrt(dt)*z)
14         total += max(st - K, 0)
```

```

5 class Portfolio:
6     name: str
7     positions: List[Option] = field(default_factory=list)
8
9     def add(self, option: Option):
10         self.positions.append(option)

```

Benefits: “Less boilerplate, immutable with frozen=True, automatic comparison.”

Q34: Type Hints

Question: Should you use type hints in Python?

3-Second Answer: “Yes! Type hints catch bugs at write-time, not runtime. Use mypy for static checking.”

Code:

```

1 from typing import List, Dict, Optional, Callable
2
3 def calculate_pnl(
4     trades: List[Dict[str, float]],
5     prices: Dict[str, float],
6     fee_model: Optional[Callable[[float], float]] = None
7 ) -> float:
8     """
9     Calculate PnL with optional fee model.
10    """
11    total = 0.0
12    for trade in trades:
13        symbol = trade['symbol']
14        qty = trade['quantity']
15        entry = trade['entry_price']
16        current = prices.get(symbol, entry)
17        pnl = qty * (current - entry)
18        if fee_model:
19            pnl -= fee_model(abs(qty * entry))
20        total += pnl
21    return total

```

Q35: Walrus Operator

Question: What is the := operator?

3-Second Answer: “Assignment expression. Assign and use value in single expression.”

Code:

```

1 # Without walrus (verbose)
2 prices = get_prices()
3 if prices:
4     process(prices)
5
6 # With walrus (concise)
7 if prices := get_prices():
8     process(prices)
9
10 # In comprehensions
11 valid_trades = [
12     (trade, pnl)
13     for trade in trades
14     if (pnl := calculate_pnl(trade)) > 0

```